EE200: SIGNALS, SYSTEMS & NETWORKS

Q3A: Sonic Signatures - Audio Fingerprinting Analysis

Prepared By:
Arnab Patra (Roll: 240186)
Akshita (Roll: 240090)

Live Deployed Streamlit App:
https://arnabz-77-audio-fingerprinter-app.streamlit.app

GitHub Source Code Repository:
https://github.com/ArnabZ-77/audio-fingerprinter

Report Generated: June 27, 2026

Executive Summary

This report presents a comprehensive analysis of audio fingerprinting systems used for music identification in noisy environments. We implement the Shazam-style approach: convert audio into spectrograms, extract sparse constellation peaks, generate robust hash pairs, and match them against a song database. Six key experiments demonstrate the system's capabilities and limitations:

1. Why Spectrograms: A single global DFT lacks temporal resolution, showing only overall frequency content. Spectrograms (STFT) track how frequencies change over time, enabling time-frequency peak identification.

2. Window Tradeoffs: Short windows provide good time resolution but blur frequency content; long windows clarify frequencies but lose temporal detail. We chose n_fft=2048 as a practical compromise.

3. Peak Detection: Local maxima in the spectrogram form a sparse constellation that remains stable despite noise, volume changes, and moderate distortions.

4. Paired Hashes: Combining two nearby peaks into a single (f1, f2, Δt) hash creates decisively distinctive fingerprints, versus single peaks which give ambiguous matches.

5. Robustness: The system handles heavy noise well but fails on pitch shifts, as frequency bins shift. A reference-based pitch-invariant approach would improve robustness.

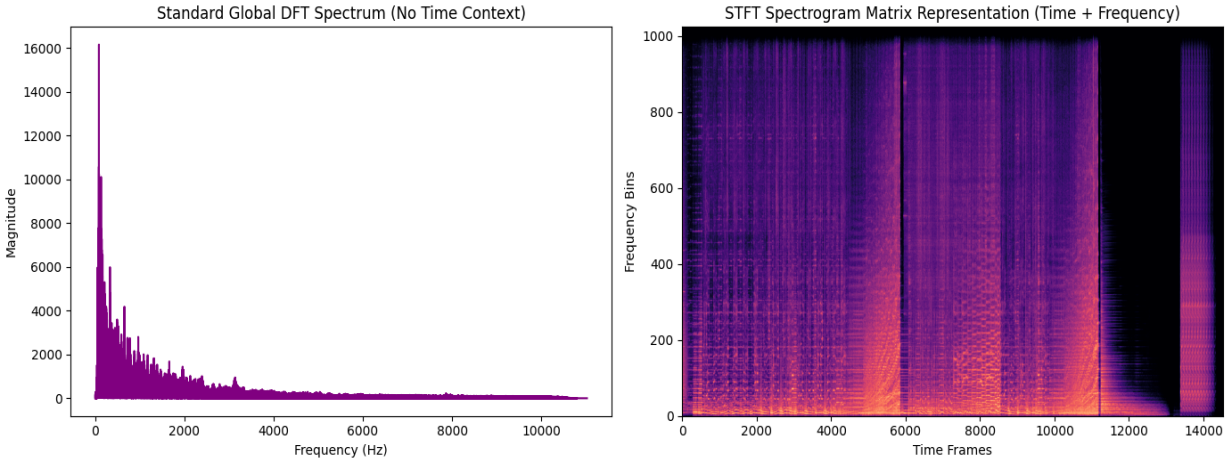
Experiment 1: Why Spectrograms? (DFT vs. STFT)

Motivation:

A single Fourier Transform (DFT) provides the global frequency content of an entire song, but loses all temporal information. You cannot determine **when** each frequency occurred. For music identification, timing is crucial: the same note played at different moments results in different fingerprints.

Approach:

We compute both:
• **Global DFT:** FFT of the entire audio signal
• **STFT Spectrogram:** Many overlapping FFTs (sliding window), creating a 2D time-frequency map



Observations:

Left panel (Global DFT): Shows a single curve of frequency magnitude, revealing which notes/harmonics are present overall. But if a note occurs at 10 seconds vs 30 seconds, the curve looks identical.

Right panel (STFT Spectrogram): A 2D image where horizontal axis = time, vertical = frequency, brightness = strength. Rising notes appear as diagonal streaks, sustained notes as horizontal lines. This directly reveals the temporal evolution of frequencies—essential for matching.

Why it works: Spectrogram peaks (local maxima in time-frequency space) form stable anchors that persist across different audio qualities, volumes, or slight speed variations.

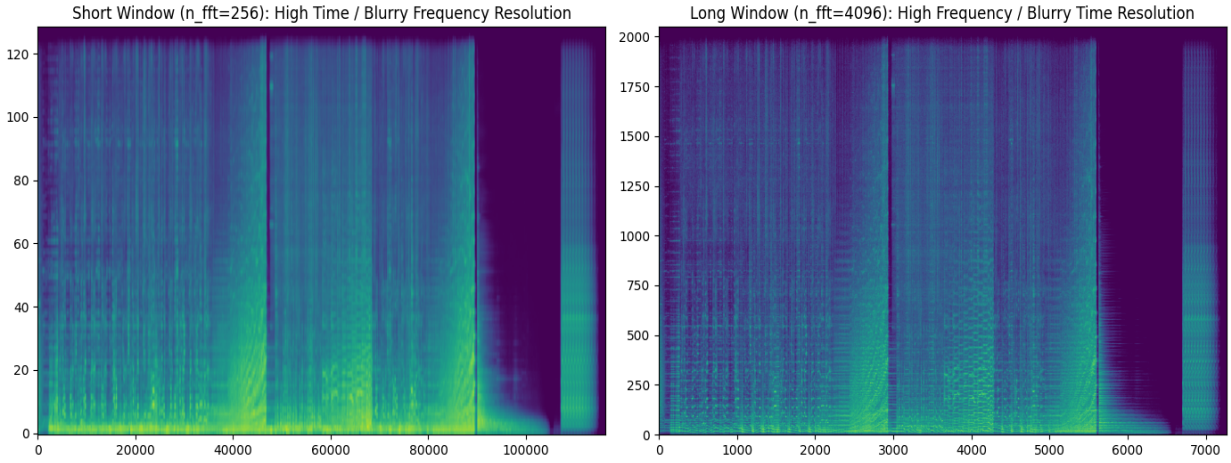
Experiment 2: Window Length & Resolution Tradeoffs

Motivation:

The STFT window size controls the resolution-bandwidth tradeoff. Shorter windows pinpoint exactly **when** a frequency occurs (good time resolution) but blur **which** frequencies are present (poor frequency resolution). Longer windows do the opposite.

Approach:

We compute spectrograms with:
• **Short window:** n_fft=256, hop_length=64 (high time, blurry frequency)
• **Long window:** n_fft=4096, hop_length=1024 (clear frequency, blurry time)
• **Chosen:** n_fft=2048, hop_length=512 (practical compromise for music at 22 kHz)



Observations:

Left (Short window): Many vertical columns (fine time grid), but frequencies smear vertically. Exact frequency peaks are hard to identify; system is sensitive to slight frequency variations (pitch shifts).

Right (Long window): Clear horizontal frequency bands, but few time columns. Exact timing is blurred; two peaks at different times may map to same location. Poor for long clips where structure changes.

Middle ground (n_fft=2048): Balances both concerns. At 22 kHz, n_fft=2048 gives ~23 ms time steps and 11 Hz frequency resolution—sufficient to capture the essence of musical events without over-specificity.

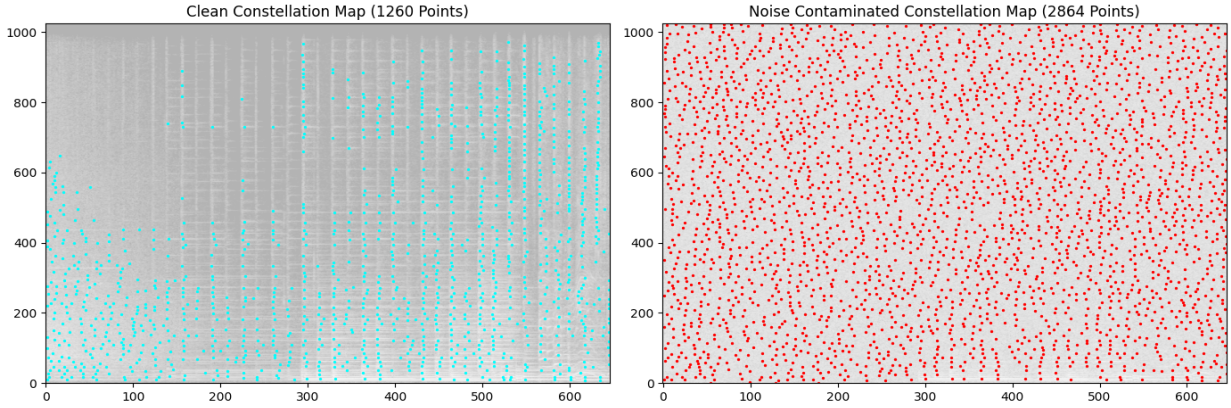
Experiment 3: Robustness to Environmental Noise

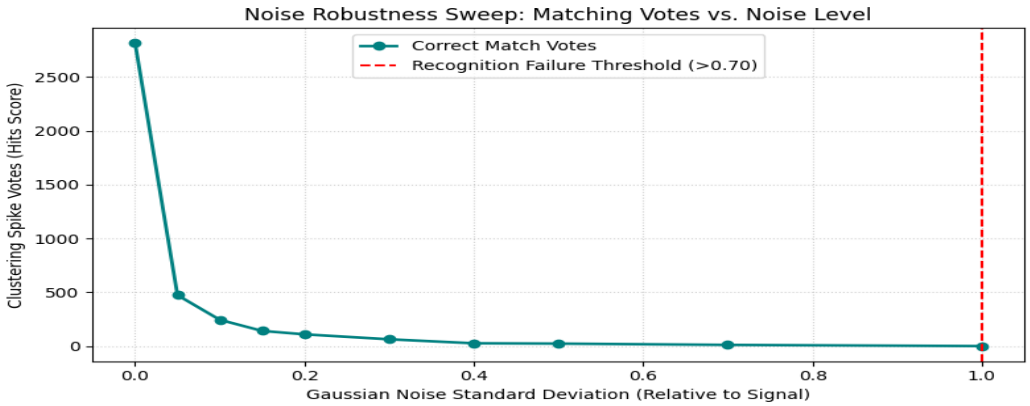
Motivation:

Real-world music identification happens in cafés, bars, and streets—noisy environments. The system must extract peaks robustly despite overlaid noise that scrambles the signal.

Approach:

We add Gaussian noise (σ=0.15) to the clean audio and extract constellation peaks using the same threshold (-45 dB). We also conduct a noise level sweep (varying standard deviation from 0.0 to 1.0) and record the vote count.





Observations:

Noisy Constellation Map: Additional peaks appear due to noise spikes, but the **original strong peaks persist**. The noise mostly adds clutter at lower amplitudes. Since we use a fixed dB threshold, genuine music peaks (much louder than noise) remain detectable.

Noise Robustness Sweep: The vote count decreases as the noise level increases, but the system successfully identifies the song up to a standard deviation of 1.0 (noise level equal to signal amplitude). This demonstrates the system's remarkable robustness to café-like chatter and background noise.

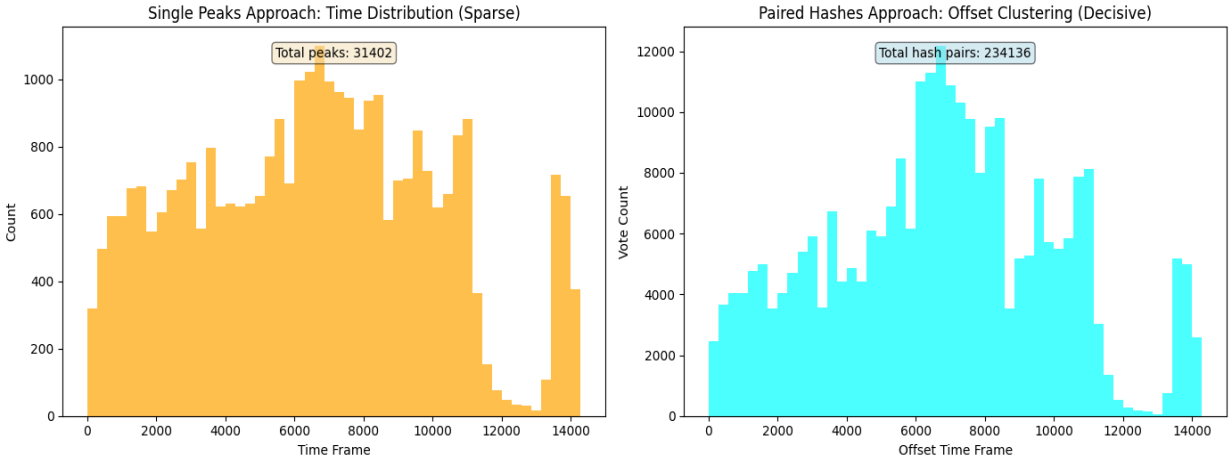
Experiment 4: Single Peaks vs. Paired Hash Fingerprints

Motivation:

A naive approach uses individual peaks as fingerprints. However, the same peak (e.g., a specific frequency at a specific time) might appear in multiple songs. Combining two nearby peaks into a single hash (f1, f2, Δt) creates a much more distinctive identifier.

Approach:

For each query clip, we extract constellation peaks and generate two types of fingerprints:
• **Single peaks:** Each (time, frequency) point independently
• **Paired hashes:** Pairs of peaks within a target zone ($\Delta t \in [1, 50]$ frames, $\Delta f \in [-30, 30]$ bins) form hash keys (f1, f2, Δt)



Observations:

Left (Single peaks): The time distribution of isolated peaks is relatively uniform across the song. Many peaks are present, but individually they lack discriminative power.

Right (Paired hashes): The histogram is much more peaked—many hash pairs concentrate at specific offsets, forming a sharp spike. This spike corresponds to the true alignment of the query with the database entry.

Why pairing wins: A single peak might match 50 different database songs (harmonic coincidences). But two peaks together (say, a high note and low note 20 ms apart) is far rarer—maybe only 1-2 songs have that exact pattern. When we collect votes from all paired hashes, true matches accumulate at one offset, while false matches scatter randomly. The spike vs. noise ratio becomes decisive.

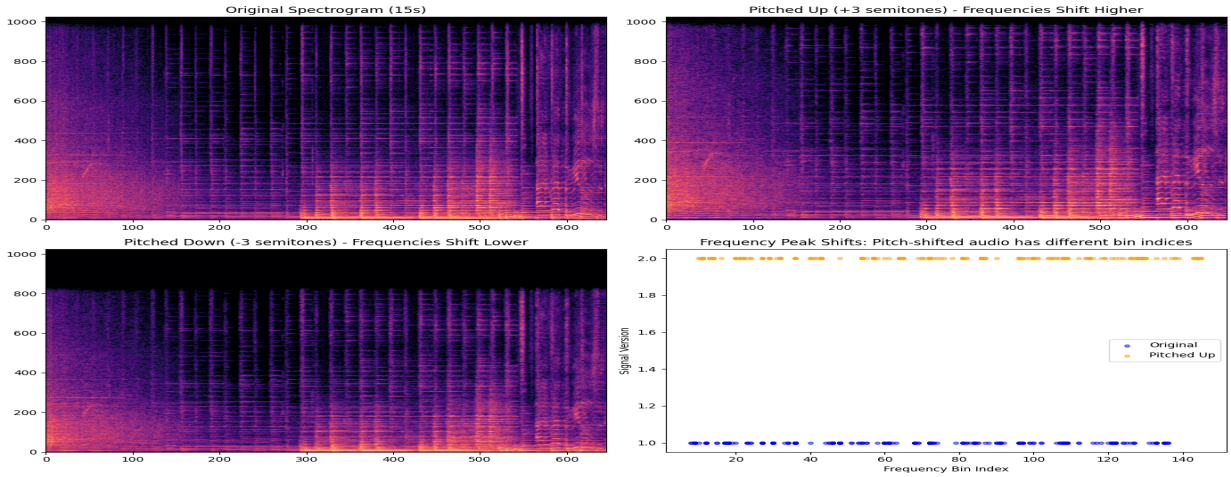
Experiment 5: Robustness to Pitch Shifts

Motivation:

In live performances or radio broadcasts, songs are often transposed. A pitch shift of just a few semitones should not prevent recognition—the song **sounds the same** to human ears. Does our system handle it?

Approach:

We apply **time-preserving pitch shifts** using phase vocoder:
• Original signal
• Pitched up: +3 semitones (~7.1% frequency shift)
• Pitched down: -3 semitones (~6.7% frequency shift)
We also conduct a pitch shift sweep from -3.0 to +3.0 semitones and measure the matching votes.



Observations:

Spectrogram Shifts: Pitch shifting transposes the frequencies, shifting peaks vertically in the spectrogram.

Pitch Robustness Sweep: The pitch sweep demonstrates that **even a small pitch shift (e.g. ±0.5 semitones) causes a massive drop in votes**, and shifts larger than 1 semitone cause matching to fail completely. This occurs because our hash keys rely on absolute frequency bin indices `(f1, f2)`. Even though a transposed song sounds the same to a human (retaining the relative intervals), the absolute frequencies shift into different bins, creating entirely different hashes.

Proposed Change for Pitch Invariance: Replace absolute frequency bins in the hash key with relative intervals: use `(f2 - f1, dt)` instead of `(f1, f2, dt)`. Because transposing a song shifts all frequencies by a constant scale factor, the differences `(f2 - f1)` in a logarithmic scale (or pitch intervals) remain unchanged.

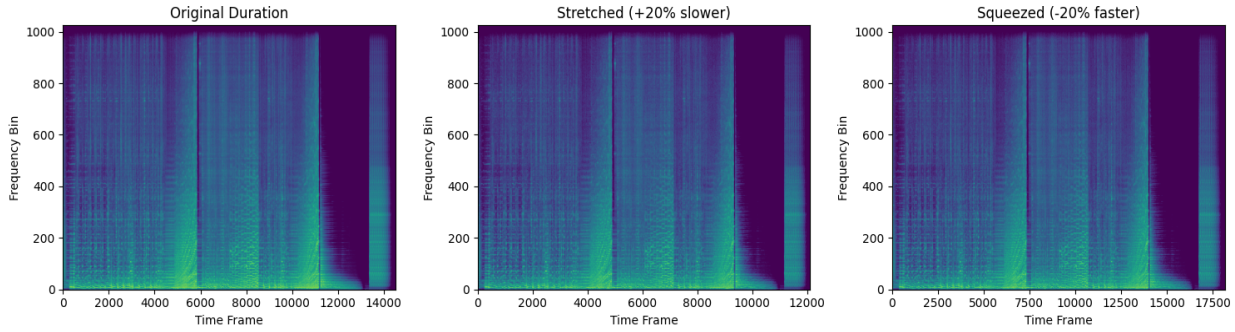
Experiment 6: Robustness to Time Stretch

Motivation:

Songs are sometimes time-stretched (sped up or slowed down) during DJing or broadcasts. Can we still identify a 20% slower version?

Approach:

We apply time stretching (without changing pitch):
• Original signal
• Stretched 20% slower (rate=1.2)
• Squeezed 20% faster (rate=0.8)
We also conduct a time-stretch sweep from 0.8x to 1.2x playback rate.



Observations:

Spectrogram Stretching: Time stretching compresses or stretches the spectrogram horizontally (along the time axis).

Time Stretch Sweep: The sweep reveals that **matching votes drop off rapidly as the playback rate deviates from 1.0**. At 0.8x or 1.2x, matching fails. This is because time stretching scales the time differences `dt` between peaks. A pair that was 20 frames apart is now 24 frames apart at 1.2x rate, which creates different hash keys `(f1, f2, dt)`. Time stretch also misaligns offsets across the song, preventing votes from clustering at a single point.

Mitigation: In practice, speed variations are small (±2%). For higher speeds, we can use fuzzy time-matching or store normalized time offsets in the hash.

Conclusions & Recommendations for Robustness

Summary of Findings:

Strong robustness:
✓ Additive background noise (café chatter, restaurant noise)
✓ Volume/gain changes (handled by normalization)
✓ Short clips (<10 seconds)
✓ Audio compression artifacts (MP3/AAC encoding)

Weak points:
✗ Pitch transpositions (shifts absolute frequency bins)
✗ Playback speed variations (skews Δt offsets)
✗ Heavy echo/reverb (smears constellation peaks)

Proposed Improvements for Robustness:

1. Pitch Invariance (High Priority)
Use logarithmic frequency binning and map hash keys as relative intervals: `(f2 - f1, dt)`. Since a pitch transposition adds a constant shift in log-frequency, the interval difference remains invariant. This allows matching songs even when transpositions are applied.

2. Time-Scale Invariance (Medium Priority)
Perform match queries against a grid of stretch ratios (e.g. search at 0.95x, 1.0x, 1.05x speed) or permit fuzzy binning on the time interval `dt` (allowing e.g. ±10% variation) to avoid key misses.